

Operating System - 550+ One-Liners

Coal India Limited - Management Trainee (Systems) | CBT Preparation

Header watermark: Asabhya Maanav

Table of Contents

OS Fundamentals - Introduction	4
OS Fundamentals - System Calls	4
Process Management - Processes	4
Process Management - Threads	5
Process Management - CPU Scheduling	5
Concurrency - IPC	6
Concurrency - Synchronization	6
Deadlocks	7
Memory Management - Main Memory	7
Virtual Memory	8
File Systems	9
Disk and Secondary Storage	9
I/O Systems	10
Protection and Security	10
CIL MT Exam Focus - Numerical Anchors	10
Additional One-Liners - Fundamentals and Structure	11
Additional One-Liners - Processes and Threads	11
Additional One-Liners - Scheduling Numerics	12
Additional One-Liners - Synchronization Depth	12
Additional One-Liners - Deadlock Depth	12
Additional One-Liners - Memory Depth	13
Additional One-Liners - Virtual Memory Depth	13

Additional One-Liners - File System Depth	14
Additional One-Liners - Disk and I/O Depth	14
Additional One-Liners - Mixed Rapid Fire	14
More One-Liners - Definitions and Numbers	15
More One-Liners - Algorithmic Complexity	16
More One-Liners - Commands and Linux Specifics	16
More One-Liners - Comparisons and Traps	16
More One-Liners - Final Numeric Drills	17
More One-Liners - Extra Coverage	18

OS Fundamentals - Introduction

- The **operating system** is system software acting as an **intermediary** between **users/applications** and **hardware**.
- The **one program** that always runs is the **kernel**; the rest are **system** and **application** programs.
- The OS has **two** classic views: the **extended/virtual machine** (convenience) and the **resource manager** (efficiency).
- The **3** main goals of an OS are **convenience**, **efficiency**, and **ability to evolve**.
- The **7** core OS functions are **process**, **memory**, **file**, **I/O**, **storage**, **protection**, and **networking** management.
- There are **2** forms of multiplexing: **time multiplexing** (CPU, printer) and **space multiplexing** (memory, disk).
- The **5** main OS types are **batch**, **multiprogramming**, **time-sharing**, **real-time**, and **distributed**.
- A **batch OS** groups **similar jobs** and optimizes **throughput** with **no user interaction**.
- A **multiprogramming OS** keeps **many jobs in memory** and optimizes **CPU utilization** by switching on **I/O wait**.
- A **time-sharing OS** uses a tiny **time quantum** to optimize **response time** for **interactive** users.
- There are **2** kinds of real-time OS: **hard** (deadline mandatory, e.g., **airbag/pacemaker**) and **soft** (best-effort, e.g., **multimedia**).
- A **distributed OS** makes **many networked machines** look like **one** system, giving **resource sharing** and **reliability**.
- The **3** parallel concepts are **multiprogramming** (1 CPU, I/O driven), **multitasking** (1 CPU, quantum driven), and **multiprocessing** (≥ 2 CPUs, true parallelism).
- Only **multiprocessing** provides **true parallelism**; the other two provide **concurrency**.
- The **4** OS structures are **monolithic**, **layered**, **microkernel**, and **hybrid**.
- A **monolithic** kernel runs everything in **kernel mode**, making it **fast** but **less reliable** (e.g., **Linux**).
- A **microkernel** keeps only **IPC**, **scheduling**, **basic memory** in the kernel and is **more reliable but slower** due to **message passing** (e.g., **QNX**, **Mach**).
- The **mode bit** has **2** values: **0 = kernel mode** and **1 = user mode**.
- **Privileged instructions** (I/O, halt, set timer, change mode bit) run **only** in **kernel mode**, else they cause a **trap**.
- The x86 architecture provides **4** protection rings: **ring 0** (kernel) to **ring 3** (user).

OS Fundamentals - System Calls

- A **system call** is the **only** legitimate entry into the **kernel** for user programs, invoked via a **trap/software interrupt**.
- There are **6** categories of system calls: **process control**, **file management**, **device management**, **information**, **communication**, and **protection**.
- Parameters can be passed to a syscall in **3** ways: **registers**, a **memory block/table**, or the **stack**.
- The **4** key process-control system calls are **fork**, **exec**, **wait**, and **exit**.
- The **4** key file system calls are **open**, **read**, **write**, and **close**.
- **fork()** returns **0 to the child**, the **child's PID to the parent**, and **-1** on failure.
- **exec()** replaces the process image and **never returns** on success, keeping the **same PID**.
- **wait()** makes the parent **block** until a child ends, collecting its **exit status** to prevent **zombies**.
- **printf()** is a **library/API** function that internally calls the **write()** system call.
- An **API** (e.g., **POSIX**, **Win32**) is preferred over raw syscalls for **portability**.

Process Management - Processes

- A **program** is **passive** (file on disk) while a **process** is **active** (program in execution).
- A process address space has **4** segments: **text**, **data**, **heap**, and **stack**.
- The **heap** grows **upward** and the **stack** grows **downward**, toward each other.
- **Local variables** live on the **stack**, **dynamic memory** on the **heap**, and **globals/statics** in the **data** segment.
- A process has **5** states: **new**, **ready**, **running**, **waiting**, and **terminated**.
- The transition **ready -> running** is performed by the **dispatcher (short-term scheduler)**.
- The transition **running -> ready** is caused by **preemption (timer interrupt)**.
- The transition **running -> waiting** is caused by an **I/O or event wait** request.
- There is **no** direct **waiting -> running** transition; a blocked process first goes to **ready**.
- The **PCB (Process Control Block)** stores **process state**, **PID**, **program counter**, **registers**, **scheduling info**, **memory info**, **accounting**, and **I/O status**.
- During a **context switch**, the CPU saves state to the **PCB** - it is **pure overhead** doing **no useful work**.
- In UNIX the first user process is **init/systemd** with **PID 1**, which **adopts orphans**.
- A **zombie** process is a **terminated child** whose **exit status** was not yet read by the parent via **wait()**.
- An **orphan** process is one whose **parent died first**; it is **adopted by init (PID 1)**.
- After **n** **fork()** calls, there are **2ⁿ** total processes and **2ⁿ - 1** children.
- The code **fork(); fork(); fork();** creates **8** total processes and **7** children.
- **Cascading termination** means terminating a **parent** forces termination of all its **children**.

Process Management - Threads

- A **thread** is the **basic unit of CPU utilization**, holding a **thread ID**, **PC**, **registers**, and **stack**.
- Threads of one process **share code**, **data/heap**, and **open files**, but keep **separate stack** and **registers**.
- The **4** benefits of threads are **responsiveness**, **resource sharing**, **economy**, and **scalability**.
- **User-level threads** are managed by a **library** and a **blocking call blocks the whole process**.
- **Kernel-level threads** are managed by the **kernel**, so one blocking thread does **not** block others.
- There are **3** multithreading models: **many-to-one**, **one-to-one**, and **many-to-many**.
- The **one-to-one** model gives the most **parallelism** but the most **kernel overhead** (e.g., **Windows/Linux**).
- The **many-to-one** model has the least overhead but **no true parallelism**.
- **Concurrency** means tasks make progress in **overlapping** time, while **parallelism** means **simultaneous** execution on **multiple cores**.
- All **parallel** execution is **concurrent**, but not all concurrency is **parallel**.

Process Management - CPU Scheduling

- There are **3** schedulers: **long-term** (job), **short-term** (CPU), and **medium-term** (swapping).
- The **long-term scheduler** controls the **degree of multiprogramming** and runs **infrequently**.
- The **short-term scheduler** runs **most frequently** (milliseconds) and must be **fast**.
- The **medium-term scheduler** performs **swapping** of processes between memory and disk.
- The **dispatcher** gives the CPU to the chosen process and introduces **dispatch latency**.
- The **5** scheduling criteria are **CPU utilization**, **throughput**, **turnaround time**, **waiting time**, and **response time**.
- **Turnaround time** = **Completion time** - **Arrival time**.
- **Waiting time** = **Turnaround time** - **Burst time**.
- **Response time** = **First CPU allocation** - **Arrival time**.
- **Non-preemptive** algorithms include **FCFS** and **SJF**; **preemptive** include **SRTF** and **Round Robin**.
- **FCFS** uses a **FIFO queue** and suffers the **convoy effect**.

- **SJF** schedules the **shortest burst** first and gives the **minimum average waiting time**.
- **SRTF** is the **preemptive** version of SJF that preempts on a **shorter-remaining-time arrival**.
- **Priority scheduling** can cause **starvation**, which is solved by **aging**.
- In most conventions, a **lower priority number** means **higher priority**.
- **Round Robin** uses a **time quantum** and becomes **FCFS** when the quantum is **very large**.
- A **very small** time quantum increases **context-switch overhead** in Round Robin.
- **Multilevel Queue** keeps processes **permanently** in one queue with **no movement**.
- **Multilevel Feedback Queue** allows processes to **move between queues** via **feedback** and **aging**.
- Burst prediction uses **exponential averaging**: $\tau(n+1) = \alpha \cdot t(n) + (1-\alpha) \cdot \tau(n)$.
- **Average waiting time** = (sum of WT) / n over n processes.

Concurrency - IPC

- The **2** fundamental IPC models are **shared memory** and **message passing**.
- **Shared memory** is the **fastest** IPC because it needs **no kernel involvement** per exchange.
- In **shared memory**, **synchronization** is the **programmer's** responsibility.
- **Message passing** uses **send()** and **receive()** and is best for **distributed** systems.
- Message passing can be **synchronous (blocking)** or **asynchronous (non-blocking)**.
- **Direct** communication names the recipient, while **indirect** uses **mailboxes/ports**.
- An **ordinary pipe** is **unidirectional** and works only between **related (parent-child)** processes.
- A **named pipe (FIFO)** persists as a **file** and works between **unrelated** processes.
- **Sockets** are endpoints identified by **IP + port** for **network** communication.
- A **signal** is an **asynchronous notification**; **SIGKILL (9)** and **SIGSTOP** cannot be **caught or ignored**.

Concurrency - Synchronization

- A **race condition** occurs when the **result depends on timing** of **concurrent** access to **shared data**.
- The **critical section** structure has **4** parts: **entry**, **critical section**, **exit**, and **remainder**.
- A correct CS solution needs **3** properties: **mutual exclusion**, **progress**, and **bounded waiting**.
- **Peterson's algorithm** solves the CS problem for **2** processes using **flag[2]** and **turn**.
- Peterson's solution uses **busy waiting** and can fail under **instruction reordering**.
- **Test-and-Set** and **Compare-and-Swap** are **atomic hardware** instructions for synchronization.
- A **semaphore** is accessed only via **2** atomic operations: **wait()/P()/down()** and **signal()/V()/up()**.
- A **counting semaphore** ranges over **0..N** while a **binary semaphore** is restricted to **0 or 1**.
- A **binary semaphore** initialized to **1** acts as a **mutex**.
- **wait = P = down = decrement**, while **signal = V = up = increment**.
- A **mutex** has **ownership** (only the locker unlocks), while a **binary semaphore** has **no ownership**.
- A **monitor** provides **automatic mutual exclusion**, allowing only **one** active process inside.
- A monitor **signal** on an empty condition variable is **lost**, unlike a semaphore's remembered **increment**.
- The bounded-buffer problem uses **3** semaphores: **mutex=1**, **empty=n**, and **full=0**.
- In bounded buffer, the **resource semaphore (empty/full)** must be acquired **before** the **mutex** to avoid deadlock.
- The basic **readers-writers** solution can cause **writer starvation**.
- The **dining philosophers** problem has **5** philosophers and **5** chopsticks, each needing **2 adjacent** chopsticks.
- Dining philosophers deadlock when **all 5** grab their **left** chopstick, forming a **circular wait**.
- One fix for dining philosophers is to allow at most **4 (n-1)** philosophers to sit at once.

- **Priority inversion** is when a **high-priority** task waits on a resource held by a **low-priority** task, fixed by **priority inheritance**.

Deadlocks

- A **deadlock** is a set of processes each **waiting for an event** only another waiting process can cause.
- There are **4** necessary conditions for deadlock: **mutual exclusion**, **hold and wait**, **no preemption**, and **circular wait**.
- All **4** deadlock conditions must hold **simultaneously** for a deadlock to occur.
- **Mutual exclusion** means at least one resource is **non-sharable**.
- **Hold and wait** means a process **holds** resources while **waiting** for more.
- **No preemption** means resources are released only **voluntarily**.
- **Circular wait** means a **closed chain** of processes each waits for the next's resource.
- In a **Resource Allocation Graph**, a **request edge** is **P -> R** and an **assignment edge** is **R -> P**.
- With **single-instance** resources, a **cycle** in the RAG means **deadlock**.
- With **multiple-instance** resources, a cycle is **necessary but not sufficient** for deadlock.
- **No cycle** in a RAG always means **no deadlock**.
- There are **4** deadlock handling strategies: **prevention**, **avoidance**, **detection and recovery**, and **ignore (ostrich)**.
- **Deadlock prevention** ensures at least **one** of the **4** conditions cannot hold.
- **Circular wait** is prevented by imposing a **total ordering** on resource types.
- **Hold and wait** is prevented by requiring a process to request **all** resources at once.
- **Deadlock avoidance** keeps the system in a **safe state** using **a priori maximum** needs.
- A state is **safe** if a **safe sequence** of all processes exists.
- A **safe** state means **no deadlock**, while an **unsafe** state means deadlock is **possible (not certain)**.
- The **Banker's algorithm** handles **multiple instances** using **Available**, **Max**, **Allocation**, and **Need** matrices.
- In the Banker's algorithm, **Need = Max - Allocation**.
- The **safety algorithm** finds a process with **Need <= Work**, frees it, and repeats until all **finish**.
- The **resource-request algorithm** grants a request only if **Request <= Need**, **Request <= Available**, and the resulting state is **safe**.
- A **wait-for graph** is built by removing **resource nodes** from the RAG (single-instance only).
- **Deadlock recovery** uses **process termination** or **resource preemption (victim + rollback)**.
- **Deadlock** blocks processes with **no state change**, **starvation** indefinitely delays a process, and **livelock** changes state with **no progress**.
- In **livelock**, processes are **active (not blocked)** but achieve **nothing**.

Memory Management - Main Memory

- **Address binding** can occur at **3** times: **compile time**, **load time**, or **execution time**.
- **Compile-time** binding produces **absolute code**; **execution-time** binding needs **MMU** hardware.
- A **logical/virtual address** is generated by the **CPU**, while a **physical address** is seen by the **memory unit**.
- The **MMU (Memory Management Unit)** translates **logical to physical** addresses at **run time**.
- Logical and physical addresses **differ** only under **execution-time** binding.
- In simple relocation, **physical address = logical address + base register**.
- The **limit register** bounds the **size**; an access **>= limit** causes a **trap**.
- **Dynamic loading** loads a routine **only when called**, improving memory use.
- **Dynamic linking** defers linking to **run time** via **stubs** and enables **shared libraries**.

- **Swapping** moves a process between **memory and disk** and is dominated by **transfer time**.
- **Contiguous allocation** gives each process a **single contiguous** block.
- **Fixed partitioning** causes **internal fragmentation** (waste inside a block).
- **Variable partitioning** causes **external fragmentation** (scattered free holes).
- There are **3** placement algorithms: **first-fit** (fastest), **best-fit** (smallest hole), and **worst-fit** (largest hole).
- **Compaction** merges scattered free holes and requires **dynamic relocation**.
- **Internal** fragmentation occurs in **fixed partitioning and paging**; **external** in **variable partitioning and segmentation**.
- **Paging** divides logical memory into **pages** and physical memory into equal-size **frames**, eliminating **external** fragmentation.
- Page size equals frame size and is a **power of 2**, typically **4 KB**.
- The **page table** maps **page number** to **frame number**, with its base in the **PTBR**.
- If page size = 2^n , the page **offset** uses **n** bits.
- The number of page-table entries = **logical address space / page size**.
- **Physical address** = **frame number * page size + offset**.
- Without a TLB, single-level paging needs **2** memory accesses per reference.
- The **TLB** is a fast **associative cache** of recent page-table entries.
- **EMAT** = $h*(c+m) + (1-h)*(c+2m)$, where h = hit ratio, c = TLB time, m = memory time.
- **Two-level paging** needs **3** memory accesses without a TLB.
- An **inverted page table** has **one entry per frame**, storing (**pid, page number**).
- Paging causes **internal** fragmentation only in the **last page**, averaging **half a page** per process.
- **Segmentation** uses **variable-size** segments matching the **user's logical view**.
- A segment table stores **base** and **limit** per segment; physical = **base + offset** if **offset < limit**.
- **Paging** is **transparent** with **internal** fragmentation; **segmentation** reflects **user view** with **external** fragmentation.
- **Segmentation with paging** removes **external** fragmentation while keeping **logical division** (e.g., Intel x86).

Virtual Memory

- **Virtual memory** lets a process run while **not fully in physical memory**, giving a large-memory illusion.
- Virtual memory is implemented mainly by **demand paging**.
- **Demand paging** loads a page **only when referenced**, using a **valid-invalid bit**.
- A **page fault** occurs when a referenced page is **not in memory** and triggers a **disk read**.
- A page fault is handled in **5** steps and ends by **restarting** the faulting instruction.
- **Pure demand paging** starts a process with **zero** resident pages.
- **EAT** = $(1-p)*\text{memory_access} + p*\text{page_fault_time}$, where p = **page-fault rate**.
- Because disk (**~ms**) is far slower than memory (**~ns**), even a tiny fault rate hurts **EAT**.
- The **dirty/modify bit** avoids writing back an **unmodified** victim page.
- There are **5+** page replacement algorithms: **FIFO**, **OPT**, **LRU**, **Clock/second-chance**, and **LFU/MFU**.
- **FIFO** replaces the **oldest** loaded page and can show **Belady's anomaly**.
- **Belady's anomaly** is when **more frames cause more faults**, occurring in **FIFO**.
- **OPT** replaces the page **used farthest in the future**, giving the **minimum** faults but is **unimplementable**.
- **LRU** replaces the **least recently used** page and is a **stack algorithm** (no Belady's anomaly).
- **Stack algorithms** like **LRU** and **OPT** never suffer **Belady's anomaly**.
- **Second-chance/clock** is FIFO using a **reference bit** to grant a **second chance**.
- **LFU** replaces the **least frequently used** page; **MFU** replaces the **most frequently used**.
- **Frame allocation** can be **equal** (m/n each) or **proportional** (by process size).

- **Global replacement** takes frames from **any** process; **local** replaces only the process's **own** frames.
- **Thrashing** is when a process spends more time **paging** than **executing**, signaled by **low CPU use**.
- **Thrashing** is caused by **too few frames / over-multiprogramming** and fixed by reducing **multiprogramming**.
- **Locality of reference** has **2** forms: **temporal** and **spatial**.
- The **working set** $W(t, \Delta)$ is the set of pages referenced in the last **Delta** references.
- A **small** page size reduces **internal** fragmentation but **enlarges the page table** and increases faults.
- A **large** page size shrinks the page table but increases **internal** fragmentation.

File Systems

- A **file** is a named collection of related information and the **smallest logical** unit of secondary storage.
- File **attributes** include **name**, **identifier**, **type**, **location**, **size**, **protection**, and **timestamps**.
- The **8** basic file operations are **create**, **write**, **read**, **seek**, **delete**, **truncate**, **open**, and **close**.
- A **file descriptor** is an **index** into the per-process **open-file table**.
- There are **3** access methods: **sequential**, **direct (relative)**, and **indexed**.
- **Sequential** access mimics **magnetic tape**; **direct** access mimics **disk** with **O(1)** block access.
- There are **5** directory structures: **single-level**, **two-level**, **tree**, **acyclic-graph**, and **general graph**.
- A **two-level** directory gives each user a **UFD** under the **MFD**, solving cross-user name conflicts.
- An **acyclic-graph** directory allows **sharing** without **cycles**, using **links**.
- A **general-graph** directory permits cycles and needs **garbage collection**.
- UNIX file protection uses **9** permission bits across **owner**, **group**, and **others**, each with **r/w/x**.
- Permission octal values are **r=4**, **w=2**, **x=1**.
- The octal **755** means **rwxr-xr-x** and **644** means **rw-r--r--**.
- The **chmod** command sets permissions, **chown** changes owner, and **umask** sets the default mask.
- There are **4** file allocation methods: **contiguous**, **linked**, **indexed**, and **inode-based**.
- **Contiguous** allocation gives the best **direct access** but causes **external** fragmentation.
- **Linked** allocation has **no external** fragmentation but **poor direct access**.
- The **FAT (File Allocation Table)** places block pointers in a **table at the volume start** to improve **random access**.
- **Indexed** allocation keeps all block pointers in an **index block**, enabling **direct access**.
- A UNIX **inode** has **12 direct**, **1 single-indirect**, **1 double-indirect**, and **1 triple-indirect** pointers.
- With block size B and pointer size P, a block holds **$k = B/P$** pointers.
- Max inode file size = **$(12 + k + k^2 + k^3) * B$** bytes.
- Single indirect adds **$k*B$** , double indirect **k^2*B** , and triple indirect **k^3*B** to file size.
- There are **4** free-space management methods: **bitmap**, **linked list**, **grouping**, and **counting**.
- A **bitmap** uses **one bit per block (0=free, 1=used)** and makes finding the first free block easy.
- The **counting** method stores **address + count** and works best for **clustered** free blocks.
- Directory implementation uses **linear list** (**O(n)** search) or **hash table** (**O(1)** average).

Disk and Secondary Storage

- A hard disk has **platters**, **tracks**, **sectors**, and **cylinders**.
- A **cylinder** is the set of **same-position tracks** across all platters.
- **Disk access time = seek time + rotational latency + transfer time**.
- **Seek time** (moving the head) is usually the **dominant** component.
- Average **rotational latency** equals time for **half a rotation** = **$0.5 * (60/\text{RPM})$** seconds.

- At **7200 RPM**, one rotation is **8.33 ms** and average latency is about **4.17 ms**.
- There are **6** main disk scheduling algorithms: **FCFS**, **SSTF**, **SCAN**, **C-SCAN**, **LOOK**, and **C-LOOK**.
- **FCFS** disk scheduling serves requests in **arrival order** (FIFO).
- **SSTF** serves the **nearest** request first and can cause **starvation**, like **SJF**.
- **SCAN (elevator)** sweeps to the **disk end** then **reverses**.
- **C-SCAN** sweeps to the end then **jumps back** without servicing, giving **uniform** wait.
- **LOOK** goes only to the **last request**, not the physical end.
- **C-LOOK** reverses at the **last request** and jumps to the **first** request.
- **Total head movement** = sum of **[consecutive position differences]** in service order.
- **SCAN/C-SCAN** reach the **physical end**; **LOOK/C-LOOK** stop at the **last request**.
- **RAID** combines disks for **performance** and/or **reliability**.
- **RAID 0** is **striping** with **no** fault tolerance.
- **RAID 1** is **mirroring** with **50%** space overhead.
- **RAID 5** uses **distributed parity** and survives **one** disk failure.
- **RAID 4** uses a **dedicated parity** disk while **RAID 5** distributes parity.
- **RAID 6** uses **double distributed parity** and survives **two** disk failures.

I/O Systems

- A device controller has **4** register types: **data-in**, **data-out**, **status**, and **control**.
- There are **2** I/O addressing schemes: **memory-mapped I/O** and **port-mapped I/O**.
- **Programmed I/O (polling)** makes the CPU **busy-wait** checking the **status bit**, wasting CPU.
- **Interrupt-driven I/O** frees the CPU using an **ISR** reached via the **interrupt vector**.
- A **non-maskable interrupt (NMI)** cannot be **deferred/masked**, unlike maskable ones.
- **DMA** transfers data **directly between device and memory**, interrupting the CPU only **once per block**.
- **Cycle stealing** is when the **DMA controller** borrows **bus cycles** from the CPU.
- **Buffering** smooths **speed mismatch**, **caching** reuses **hot data**, and **spooling** queues output for a **printer**.
- A **cache** holds a **copy** of data, while a **buffer** may hold the **only** copy in transit.

Protection and Security

- **Protection** enforces the **principle of least privilege** on resource access.
- The **access matrix** has **rows = domains** and **columns = objects**.
- An **Access Control List (ACL)** stores the matrix by **column (per object)**.
- A **capability list** stores the matrix by **row (per domain)**.
- There are **3** authentication factors: **something you know**, **something you have**, and **something you are**.
- Passwords should be stored as **salted hashes**, never **plaintext**.
- **Multi-factor authentication (MFA)** combines **two or more** authentication factors.

CIL MT Exam Focus - Numerical Anchors

- **Turnaround = Completion - Arrival** and **Waiting = Turnaround - Burst**.
- **SJF** gives the **minimum** average waiting time among scheduling algorithms.
- **Need = Max - Allocation** in the Banker's algorithm.
- **Belady's anomaly** appears in **FIFO** but never in **LRU/OPT**.
- **OPT** gives the **theoretical minimum** page faults but is **unimplementable**.

- **EMAT** = $h \cdot (c+m) + (1-h) \cdot (c+2m)$ for single-level paging with a TLB.
- **Physical address** = **frame number * page size + offset**.
- Average rotational latency = **half a rotation**.
- Max inode file size = $(12 + k + k^2 + k^3) \cdot B$ with $k = B/P$.
- After **n** forks there are **2^n** processes and **2^n - 1** children.
- The **5** process states are **new, ready, running, waiting, terminated**.
- The **2** fragmentation types are **internal** and **external**.
- The **4** deadlock conditions are **mutual exclusion, hold-and-wait, no preemption, circular wait**.
- The **3** critical-section requirements are **mutual exclusion, progress, bounded waiting**.
- The **fastest IPC** mechanism is **shared memory**.
- The **uncatchable** signals are **SIGKILL (9)** and **SIGSTOP**.
- The **mode bit** convention is **0 = kernel** and **1 = user**.

Additional One-Liners - Fundamentals and Structure

- The **kernel** runs in **kernel mode** while application code runs in **user mode**, separated by the **mode bit**.
- A **trap** is a **software-generated** interrupt caused by an **error** or a **system-call** request.
- An **interrupt** is a **hardware** signal that diverts the CPU to an **interrupt handler** via the **interrupt vector**.
- The **bootstrap program** (firmware in **ROM/BIOS/UEFI**) loads the **kernel** at power-on.
- **Spooling** historically replaced **offline** batch processing by overlapping **I/O with computation**.
- A **layered OS** has layer **0 = hardware** and the top layer = the **user interface**.
- The main drawback of a **layered OS** is **performance overhead** from traversing layers.
- **Loadable kernel modules (LKM)** let modern OSes add functionality **at runtime** (e.g., **Linux**).
- **SMP (symmetric multiprocessing)** treats all CPUs as **peers**, while **asymmetric** has one **master**.
- The **degree of multiprogramming** is the **number of processes in memory** simultaneously.
- **Throughput** is measured as the number of **processes completed per unit time**.
- A **real-time** system is judged by **meeting deadlines**, not by average speed.
- **Caching** works because of **locality of reference** in access patterns.
- The **shell/command interpreter** is a **system program**, not part of the **kernel**.
- A **virtual machine** abstracts hardware so multiple **guest OSes** run on one host via a **hypervisor**.

Additional One-Liners - Processes and Threads

- The **ready queue** holds processes that are **ready and waiting** for the CPU.
- **Device/I-O queues** hold processes **waiting** for a particular device.
- A process is **CPU-bound** if it spends most time **computing**, and **I/O-bound** if mostly **waiting on I/O**.
- Good scheduling mixes **CPU-bound and I/O-bound** processes to keep both CPU and devices **busy**.
- The **copy-on-write** technique lets **fork()** share pages until one process **writes**, saving memory.
- A **vfork()** suspends the parent and shares its address space until the child calls **exec** or **exit**.
- A **thread pool** reuses a **fixed number** of worker threads to avoid repeated **creation overhead**.
- The **TCB (Thread Control Block)** stores per-thread state analogous to the **PCB** for processes.
- **Thread cancellation** can be **asynchronous** (immediate) or **deferred** (at cancellation points).
- A **lightweight process (LWP)** is an intermediate structure between **user and kernel** threads.
- The **program counter** of a process points to the **next instruction** to execute.
- A **process tree** in UNIX is rooted at **init/systemd (PID 1)**.
- The **exit status** returned by a child is collected by the parent through **wait()/waitpid()**.

- A **daemon** is a **background** process, often a **child of init**, providing services.
- The **nice value** in UNIX adjusts a process's **scheduling priority**.

Additional One-Liners - Scheduling Numerics

- In **FCFS**, if a long job arrives first, short jobs suffer the **convoy effect**, raising **average waiting time**.
- **SJF** requires knowing the **next CPU burst**, often estimated by **exponential averaging**.
- **SRTF** re-evaluates the schedule at every **new arrival** based on **remaining time**.
- **Round Robin** with quantum q and n processes bounds waiting to about $(n-1)*q$ per cycle.
- The **number of context switches** in Round Robin **increases** as the time quantum **decreases**.
- **Preemptive priority** scheduling preempts when a **higher-priority** process arrives.
- **Aging** prevents **starvation** by gradually **raising** the priority of waiting processes.
- The **dispatch latency** is the time to **stop one process and start another**.
- **CPU utilization** is theoretically $1 - p^n$ for n processes each spending fraction p on I/O.
- The **response ratio** in HRRN scheduling is $(\text{waiting} + \text{burst}) / \text{burst}$.
- **HRRN (Highest Response Ratio Next)** is **non-preemptive** and reduces **starvation** of long jobs.
- In a tie during **SJF**, the usual rule is to apply **FCFS** order.
- A **Gantt chart** reads each process's **completion time** from the timeline.
- **Idle CPU time** appears in a Gantt chart when **no process has arrived** yet.
- The **average turnaround time** equals the **sum of TAT** divided by n .

Additional One-Liners - Synchronization Depth

- A **spinlock** is a lock where a thread **busy-waits** in a loop until it is free.
- **Busy waiting** wastes CPU but avoids the cost of a **context switch** for short critical sections.
- A **deadlock** in semaphores can arise from acquiring locks in **inconsistent order**.
- **Bounded waiting** guarantees a **limit** on how many times others enter before a waiter.
- **Progress** ensures that only processes **not in their remainder** decide who enters next, with no **indefinite postponement**.
- A **counting semaphore** initialized to N permits up to N concurrent accesses.
- The **V (signal)** operation may **wake a blocked process** from the semaphore's queue.
- **Two** processes using Peterson's algorithm coordinate via **turn** giving away priority.
- The **test-and-set** instruction sets a lock to **true** and returns its **old value** atomically.
- **Compare-and-swap** succeeds only if the variable equals the **expected** value.
- A **condition variable** supports **wait** and **signal**, used inside a **monitor**.
- **Hoare** monitors transfer control immediately on **signal**, while **Mesa** monitors re-check the condition.
- The producer in bounded buffer first does **wait(empty)** then **wait(mutex)**.
- The consumer in bounded buffer first does **wait(full)** then **wait(mutex)**.
- Reversing **mutex** and **resource** semaphores in bounded buffer causes a **deadlock**.
- **Readers-writers** variants include **reader-priority**, **writer-priority**, and **fair**.
- A **recursive mutex** can be **re-locked** by the same thread without deadlock.
- **Atomic** operations are **indivisible** - they complete fully or not at all.
- A **memory barrier/fence** prevents harmful **instruction reordering** around critical sections.
- **Lock contention** rises as more threads compete for the **same lock**.

Additional One-Liners - Deadlock Depth

- The **ostrich algorithm** simply **ignores** deadlocks, used when they are **rare** (e.g., UNIX/Windows).
- **Deadlock avoidance** is less restrictive than **prevention** but needs **advance information**.
- A **single-instance** system can use the **resource-allocation-graph** algorithm with **claim edges**.
- A **claim edge** $P \rightarrow R$ indicates P **may request** R in the future.
- In the Banker's algorithm, **Work** is initialized to **Available**.
- The **Finish** vector starts **false** and becomes **true** as processes are simulated to finish.
- If the safety algorithm marks **all** processes $Finish=true$, the state is **safe**.
- A request exceeding **Need** signals an **error** in the resource-request algorithm.
- A request exceeding **Available** makes the process **wait**.
- **Resource preemption** recovery must address **victim selection**, **rollback**, and **starvation**.
- **Total ordering** of resources prevents the **circular wait** condition.
- The number of deadlock conditions that must hold together is **4**.

Additional One-Liners - Memory Depth

- **Internal** fragmentation is wasted space **inside** an allocated region.
- **External** fragmentation is free space **scattered** between allocations.
- The **50-percent rule** states first-fit may waste about **half** as many blocks as allocated to fragmentation.
- **Best-fit** must search the **entire** free list unless it is sorted.
- **Worst-fit** tends to leave the **largest usable** remaining hole.
- The **PTBR (page-table base register)** points to the **page table** in memory.
- The **PTLR (page-table length register)** bounds the **page-table size**.
- A **valid-invalid bit** in a page-table entry marks whether the page is **legal/resident**.
- **Protection bits** in a page-table entry control **read/write/execute** access.
- Page-table entries also store a **dirty bit** and a **reference bit**.
- The **frame number** concatenated with the **offset** gives the **physical address**.
- A **TLB hit** avoids the extra **memory access** for the page table.
- **ASIDs (address-space identifiers)** let a TLB hold entries for **multiple processes**.
- **Hashed page tables** are common for address spaces larger than **32 bits**.
- **Segmentation** can share a segment by pointing **two segment-table entries** to the same base.
- A segmentation **limit** violation raises a **trap to the OS**.
- In **x86**, logical-to-linear uses **segmentation**, then linear-to-physical uses **paging**.

Additional One-Liners - Virtual Memory Depth

- **Demand paging** reduces I/O by loading **only needed** pages.
- A **minor page fault** is resolved from a page already in memory (e.g., in the **page cache**).
- A **major page fault** requires a **disk read**.
- **Prepaging** loads pages **before** they are referenced to reduce future faults.
- **Page-fault frequency (PFF)** controls allocation by targeting an acceptable **fault rate**.
- **Thrashing** causes the CPU scheduler to add **more processes**, worsening the problem.
- The **working-set window** Δ determines which pages are in the **working set**.
- If the sum of **working-set sizes** exceeds total frames, **thrashing** occurs.
- **FIFO** is the only common replacement algorithm that exhibits **Belady's anomaly**.
- **LRU** can be implemented with **counters** or a **stack** of page numbers.
- The **clock** algorithm uses a **circular** list and a **reference bit**.

- **Enhanced second-chance** uses the (**reference, modify**) bit pair, giving **4** classes.
- **OPT** is also called the **MIN** algorithm.
- **Demand paging** EAT degrades sharply because disk access is about **10⁵** times slower than memory.
- The **copy-on-write** optimization defers page copying until a **write**.

Additional One-Liners - File System Depth

- A **file control block (FCB)** / **inode** stores file **metadata** on disk.
- The **per-process open-file table** and the **system-wide open-file table** together track **open files**.
- **Sequential** access supports **read_next** and **write_next** operations.
- **Indexed** access keeps a **logical index** that maps keys to **block locations**.
- A **hard link** points to the same **inode**; deleting one keeps the file until the **link count** hits **0**.
- A **symbolic (soft) link** stores a **path** and can **dangle** if the target is removed.
- **Contiguous** allocation directory entries store the **start block** and **length**.
- **Linked** allocation stores the **first block** pointer in the directory.
- **Indexed** allocation can chain or use a **multilevel** index for large files.
- A **12-direct**-pointer inode covers small files with **no indirection**.
- The **single-indirect** block points to a **block of pointers** to data blocks.
- The **triple-indirect** block extends maximum file size by **k³** blocks.
- A **bitmap** free-list needs about **disk_size / block_size** bits.
- The **grouping** method stores **n free-block addresses** in the first free block.
- **Journaling** file systems log changes to recover quickly after a **crash**.

Additional One-Liners - Disk and I/O Depth

- **Logical block addressing (LBA)** numbers sectors **linearly**, replacing **CHS**.
- The **disk bandwidth** is the **bytes transferred / total time** from request to completion.
- **SSTF** is analogous to **SJF** and is **not optimal** and can **starve** requests.
- **SCAN** services requests in **both** directions of a sweep.
- **C-SCAN** provides more **uniform** waiting than **SCAN** by servicing in **one** direction.
- The **boot block** is at a **fixed** location used by the **bootstrap**.
- **Bad-block** management uses **sector sparing/forwarding** to remap defective sectors.
- **Polling** repeatedly checks the **busy bit** of a device.
- An **ISR (interrupt service routine)** runs in response to a device **interrupt**.
- **DMA** reduces CPU involvement to **setup** and a **completion interrupt**.
- **Double buffering** uses **two** buffers to overlap **I/O and processing**.
- **Spooling** is essential for **printers**, which are **non-sharable** devices.
- **RAID 0** maximizes **performance** and capacity but offers **zero** redundancy.
- **RAID 1+0 (10)** combines **mirroring and striping** for performance and reliability.
- **Parity** in RAID allows **reconstruction** of a failed disk's data.

Additional One-Liners - Mixed Rapid Fire

- The **OS** provides **abstraction**, **resource management**, and **protection**.
- A **system call** causes a **user-to-kernel** mode switch via a **trap**.
- **fork** returns **twice** while **exec** returns **zero** times on success.

- A **zombie** occupies only a **PCB entry** until **reaped**.
- **Thrashing** is detected by **high paging activity** with **low CPU utilization**.
- **SJF** and **OPT** are both **optimal** but require **future/burst knowledge**.
- **Belady's anomaly** is exclusive to **non-stack** algorithms like **FIFO**.
- The **Banker's algorithm** is named for how a **banker** avoids over-lending **cash**.
- A **cycle** with single-instance resources is **sufficient** for deadlock; with multiple instances it is **not**.
- **Mutex** ownership means only the **locking thread** may **unlock**.
- The **TLB** is also called the **associative memory** for address translation.
- **Demand paging** needs the ability to **restart** any interrupted instruction.
- **Compaction** is impossible under **static (compile-time)** relocation.
- A **page** is a **logical** unit; a **frame** is a **physical** unit of the same size.
- The **inode** number uniquely identifies a file within a **filesystem**.
- **chmod 700** grants **rwX** to the owner and **nothing** to group/others.
- **SIGSEGV** is raised on an **invalid memory access (segmentation fault)**.
- **SIGTERM (15)** requests graceful termination and **can** be caught, unlike **SIGKILL (9)**.
- A **context switch** time is counted as **overhead**, not useful work.
- The **first-fit** algorithm typically performs **better** than worst-fit for utilization and speed.

More One-Liners - Definitions and Numbers

- An OS divides resources via **2** schemes: **time-sharing** and **space-sharing**.
- The **batch** system introduced the concept of **automatic job sequencing**.
- **Multiprogramming** requires **memory protection** and **CPU scheduling** to coexist safely.
- The **timer interrupt** enables **preemption** in time-sharing systems.
- A **trap vector / interrupt vector** table maps each event to its **handler address**.
- The **system-call interface** hides implementation behind **numbered** entries.
- **POSIX** is a **standard API** for UNIX-like systems.
- **Win32** is the native **API** of **Windows**.
- The **three** categories of state for a job are **new/admitted**, **running/ready**, and **exit**.
- A process becomes a **zombie** after **exit()** and before the parent's **wait()**.
- The **init** process never has a **zombie** child because it always **reaps** them.
- A **context switch** requires saving **registers**, **PC**, and **memory maps**.
- **Multithreading** improves **throughput** on **multicore** CPUs.
- The **many-to-many** model multiplexes **m user threads** onto **n <= m kernel threads**.
- A **race condition** is eliminated by enforcing **mutual exclusion**.
- The **critical-section** problem requires **atomic** entry/exit coordination.
- **Peterson's** solution does **not** require special **hardware instructions**.
- A **binary semaphore** and a **mutex** both provide **mutual exclusion**.
- The **P** operation may **block** a process; the **V** operation may **unblock** one.
- **Deadlock** requires all **4** conditions; removing **any one** prevents it.
- The **circular wait** is the condition most easily broken by **resource ordering**.
- A **safe sequence** guarantees every process can **complete** eventually.
- **Unsafe** does not imply **deadlock**, but **deadlock** implies **unsafe**.
- **Page replacement** is needed only when **all frames are full**.
- **LRU** approximates **OPT** by using **recent past** as a predictor.
- **FIFO** replacement ignores **usage frequency** and **recency**.

- The **reference bit** supports **clock/second-chance** replacement.
- **Thrashing** reflects a violation of the **locality** assumption due to too few **frames**.
- **Demand paging** treats memory as a **cache** for the **disk**.
- The **page size** trade-off balances **table size** against **internal fragmentation**.

More One-Liners - Algorithmic Complexity

- A **linear-list** directory search has time complexity **$O(n)$** .
- A **hash-table** directory search has average complexity **$O(1)$** .
- **Best-fit** placement scanning an unsorted free list is **$O(n)$** per allocation.
- The **safety algorithm** runs in **$O(m * n^2)$** for n processes and m resource types.
- **SSTF** selecting the nearest request scans all pending requests in **$O(k)$** .
- **LRU** with a stack updates in **$O(1)$** amortized but needs extra **hardware/list** maintenance.
- **Counting sort**-like **bitmap** scan for a free block is **$O(\text{disk_blocks})$** worst case.
- A **two-level** page table lookup without TLB costs **3** memory accesses.
- A **three-level** page table without TLB costs **4** memory accesses.
- The inode max-size sum **$12 + k + k^2 + k^3$** is dominated by the **triple-indirect** term.

More One-Liners - Commands and Linux Specifics

- The **ps** command lists current **processes**.
- The **top** command shows live **CPU and memory** usage per process.
- The **kill -9 PID** command sends **SIGKILL** to force-terminate a process.
- The **nice** and **renice** commands adjust process **priority**.
- The **fork()** system call in C returns a **pid_t** value.
- The **exec** family includes **execl**, **execv**, **execvp**, and **execvp**.
- The **chmod 755 file** command sets permissions to **rwxr-xr-x**.
- The **umask 022** default yields new files with **644** and directories with **755**.
- The **df** command reports **free disk space** per filesystem.
- The **du** command reports **disk usage** of files/directories.
- The **mkfifo** command creates a **named pipe (FIFO)**.
- The **pipe()** system call creates an **anonymous pipe** with **2** file descriptors.
- The **wait()** call blocks until **any child** terminates, while **waitpid()** targets a specific child.
- The **getpid()** call returns the **current process ID**.
- The **ln** command creates a **hard link**, and **ln -s** creates a **symbolic link**.
- The **mount** command attaches a filesystem to a **mount point**.
- The **ulimit** command shows or sets process **resource limits**.
- The **free** command displays **memory and swap** usage.
- The **vmstat** command reports **virtual-memory** statistics including **paging**.
- The **iostat** command reports **disk I/O** statistics.

More One-Liners - Comparisons and Traps

- **Multiprogramming** maximizes **CPU utilization**; **time-sharing** minimizes **response time**.
- **Internal** fragmentation belongs to **fixed/paging**; **external** to **variable/segmentation**.
- **Paging** is **transparent** to the user; **segmentation** matches the **user view**.

- **Shared memory** is **faster**; **message passing** is **easier and distributed**.
- **Mutex** has **ownership**; **semaphore** does **not**.
- **Monitor** signal is **lost if no waiter**; **semaphore** signal is **remembered**.
- **SCAN** hits the **disk end**; **LOOK** stops at the **last request**.
- **C-SCAN/C-LOOK** do **not service** on the return sweep, giving **uniform** wait.
- **RAID 0** = **striping (no redundancy)**; **RAID 1** = **mirroring**.
- **RAID 4** = **dedicated parity**; **RAID 5** = **distributed parity**.
- **Long-term** scheduler controls **multiprogramming degree**; **short-term** runs **most often**.
- **Zombie** = **dead child unreaped**; **orphan** = **dead parent**.
- **FCFS** = **convoy effect**; **SSTF** = **starvation**.
- **SJF** = **minimum avg waiting**; **RR** = **best response time**.
- **OPT** = **minimum faults**; **FIFO** = **Belady's anomaly**.
- **Deadlock** = **blocked**; **livelock** = **active but stuck**.
- **Contiguous** = **best access, external frag**; **linked** = **poor access, no external frag**.
- **Hard link** shares an **inode**; **soft link** stores a **path**.
- **Polling** wastes CPU; **interrupts** free it; **DMA** offloads **bulk transfer**.
- **ACL** is **per-object**; **capability** is **per-domain**.

More One-Liners - Final Numeric Drills

- The CPU has **2** basic operating modes, **kernel** and **user**.
- There are **3** address-binding times: **compile**, **load**, and **execution**.
- There are **4** segments in a process image: **text**, **data**, **heap**, **stack**.
- There are **5** states in the process model.
- There are **6** categories of system calls.
- There are **6** disk scheduling algorithms in the standard syllabus.
- There are **4** file allocation methods.
- There are **4** free-space management techniques.
- There are **3** multithreading models.
- There are **3** CPU schedulers by time horizon.
- There are **4** necessary conditions for deadlock.
- There are **3** requirements for a critical-section solution.
- There are **2** types of semaphores: **counting** and **binary**.
- There are **2** fragmentation types: **internal** and **external**.
- There are **12** direct block pointers in a classic UNIX inode.
- There are **3** indirect pointer levels: **single**, **double**, **triple**.
- There are **2** IPC models: **shared memory** and **message passing**.
- There are **2** pipe types: **ordinary** and **named (FIFO)**.
- There are **3** authentication factors: **know**, **have**, **are**.
- There are **6** common RAID levels referenced: **0, 1, 2, 3, 4, 5**.
- There are **2** I/O addressing methods: **memory-mapped** and **port-mapped**.
- There are **3** disk-access time components: **seek**, **rotational latency**, **transfer**.
- There are **5** scheduling criteria optimized by the CPU scheduler.
- There are **3** classic synchronization problems: **producer-consumer**, **readers-writers**, **dining philosophers**.
- There are **4** deadlock-handling strategies: **prevention**, **avoidance**, **detection-recovery**, **ignore**.
- There are **5+** page replacement algorithms: **FIFO**, **OPT**, **LRU**, **clock**, **LFU/MFU**.

- There are **3** placement strategies in variable partitioning: **first-fit**, **best-fit**, **worst-fit**.
- There are **2** thread levels: **user-level** and **kernel-level**.
- There are **2** access-matrix implementations: **ACL** and **capability list**.
- There are **2** replacement scopes: **global** and **local**.

More One-Liners - Extra Coverage

- The **turnaround time** of a process is always **$\geq$ its burst time**.
- The **waiting time** of the first FCFS process (arrival 0) is **0**.
- A **negative** waiting time in a numerical indicates an **arithmetic error**.
- **Response time** matters most for **interactive/time-sharing** systems.
- The **convoy effect** disappears if the **shortest** job is scheduled first.
- **Preemption** can introduce **race conditions** on shared kernel data.
- The **dispatcher** is invoked during every **context switch**.
- A **higher** TLB hit ratio **lowers** the effective memory access time.
- A **page fault** during instruction fetch must allow the instruction to be **restarted**.
- The **modify (dirty) bit** being **0** lets the OS skip writing the victim page to disk.
- **Demand paging** with **$p = 0$** reduces EAT to the plain **memory access time**.
- A **larger** working set requires **more** allocated frames to avoid thrashing.
- The **bitmap** method makes **contiguous** free-block allocation easy.
- A **symbolic link** can cross **filesystem** boundaries, unlike a **hard link**.
- The **root directory** in UNIX is denoted by **/**.
- **Mounting** attaches a device's filesystem at a **directory (mount point)**.
- The **seek time** dominates total disk access for **random** workloads.
- **Sequential** disk workloads minimize **seek** overhead.
- A **DMA** transfer frees the CPU to run **other processes** during the transfer.
- **Spooling** lets multiple jobs queue output to a **single printer**.
- The **access matrix** can be sparse, motivating **ACL** or **capability** storage.
- **Least privilege** limits each domain to the **minimum** rights needed.
- **Salting** a password defends against **precomputed (rainbow-table)** attacks.
- A **context switch** count rises with a **smaller** Round Robin quantum.
- The number of children after **4** forks is **15 ($2^4 - 1$)**.
- A **counting semaphore** for **3** identical printers is initialized to **3**.
- The **empty** semaphore in a buffer of size **10** is initialized to **10**.
- The **full** semaphore in any bounded buffer starts at **0**.
- A **deadlock** of **2** processes can form over **2** resources via **circular wait**.
- The **wait-for graph** applies only to **single-instance** resource systems.